

Conversa completa Projeto KIZUNA

Transcrição integral, do início ao fim

03 de setembro de 2026

[Turno 1](#)

[Turno 2](#)

[Turno 3](#)

[Turno 4](#)

[Turno 5](#)

[Turno 6](#)

[Turno 7](#)

[Turno 8](#)

Turno 1

Wilson

sera que conseguiríamos criar uma versao melhorada do Turbo Pascal 3 para o MSX? ou ate do Turbo Pascal 4? rodando em PC, mas gerando arquivos .COM para o MSX com Z80, MSX 2, 128Kb de RAM, MSX-DOS ?

Claude

Sim, é totalmente viável — e na verdade existe um precedente histórico interessante: o **Turbo Pascal original (v1-3) já rodava em CP/M-80 com Z80**, e o **MSX-DOS 1 é compatível a nível de BDOS com CP/M-80**. Então tecnicamente muitos .COM de Turbo Pascal 3 para CP/M já rodam (ou quase rodam) num MSX-DOS 1. O TP4 já era exclusivo para MS-DOS/x86, então “recriar” ele para Z80 seria mais um trabalho de reinterpretação de linguagem do que port.

Alguns pontos técnicos que já valem antecipar:

Arquitetura do compilador (cross-compiler em Go, rodando no PC):

- Frontend: lexer/parser/AST para um subset de Object Pascal clássico (bem dentro do que você já manja em Go)
- Backend: gerador de código Z80 — aqui a decisão principal é se você escreve seu próprio assembler/linker Z80 do zero (mais controle, mais trabalho) ou gera assembly textual e usa o toolchain do **SDCC** (`sdasz80/sdldz80`) como back-end, que você já está usando no projeto MSXgl
- Runtime: I/O via chamadas de BDOS do MSX-DOS (compatíveis com CP/M) para arquivos, e BIOS/VRAM direto para tela — MSX2 dá telas melhores que o CP/M puro que o TP original usava

Coisa que muda o jogo, memória: TP3 original em CP/M ficava limitado a ~60KB de TPA (tudo — código + dados — num segmento só). No MSX2 com 128KB e memory mapper, dá pra ultrapassar isso com bank switching, permitindo programas bem maiores que o original — aí sim ficaria “melhor que o Turbo Pascal 3” de verdade.

Prior art que vale olhar antes de começar: existe o **Turbo Rascal Syntax Error (8-bit Edition)**, um compilador Pascal-like moderno que já gera código para Z80/MSX entre outras plataformas 8-bit — vale estudar as decisões de design dele mesmo que você não re-use código.

Antes de eu detalhar uma arquitetura concreta, isso muda bastante dependendo de algumas escolhas:

[Pergunta de esclarecimento 1] Escopo da linguagem: você quer replicar fielmente o Pascal do TP3/TP4 (records, sets, ponteiros, units) ou um subset enxuto pra começar a rodar algo logo? Opções: Subset mínimo primeiro (tipos básicos, procedures, arrays) / Fiel ao TP3 (sem units) / Fiel ao TP4 (com units/modularização) / Ainda não sei, quero discutir

[Pergunta de esclarecimento 2] Back-end Z80: prefere escrever seu próprio assembler/linker em Go do zero, ou gerar assembly e usar o toolchain do SDCC (`sdasz80/sdldz80`) que você já usa no projeto MSXgl? Opções: Assembler/linker próprio em Go / Reusar SDCC (`sdasz80/sdldz80`) / Ainda não decidi

[Pergunta de esclarecimento 3] Modelo de memória alvo: ficar compatível com o limite clássico (~60KB, roda em qualquer MSX-DOS) ou já projetar pra usar os 128KB com memory mapper do MSX2? Opções: Compatível clássico (~60KB) / Usar 128KB com bank switching desde o início / Suportar os dois modos

Turno 2

Wilson

P: Escopo da linguagem: você quer replicar fielmente o Pascal do TP3/TP4 (records, sets, ponteiros, units) ou um subset enxuto pra começar a rodar algo logo? R: Fiel ao TP4 (com units/modularização)

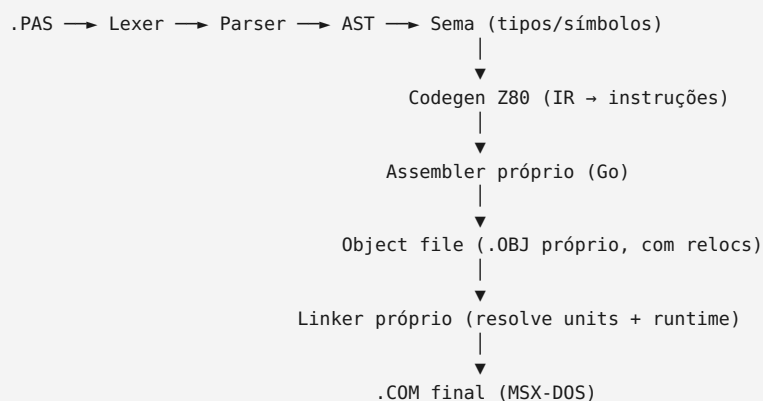
P: Back-end Z80: prefere escrever seu próprio assembler/linker em Go do zero, ou gerar assembly e usar o toolchain do SDCC (sdasz80/slddz80) que você já usa no projeto MSXgl? R: Assembler/linker próprio em Go

P: Modelo de memória alvo: ficar compatível com o limite clássico (~60KB, roda em qualquer MSX-DOS) ou já projetar pra usar os 128KB com memory mapper do MSX2? R: Usar 128KB com bank switching desde o início

Claude

Boa, essas três escolhas juntas (TP4 completo + assembler/linker próprio + mapper de 128KB desde o início) formam um projeto ambicioso mas bem definido. Vou desenhar a arquitetura em blocos.

1. Pipeline geral



Tudo isso roda no PC (Go), só o .COM de saída é Z80.

2. Sistema de units (equivalente ao .TPU do TP4)

Cada unit compila para um objeto intermediário próprio contendo:

- **Interface section:** símbolos exportados (tipos, procedures/functions, vars) com assinatura serializada
- **Implementation section:** código já montado em bytecode/relocs, ainda não linkado a endereço final
- **Tabela de dependências:** quais outras units ela `uses`

O compilador, ao processar `uses UnitX`, lê só a interface (rápido, sem reparsear implementação) — exatamente como o TP4 fazia. Isso significa você precisa de um **formato de arquivo de unit compilada** (posso ajudar a desenhar o layout binário quando chegar a hora).

3. Assembler/linker próprio em Go

Como você optou por não usar SDCC, isso implica:

- **Tabela de encoding Z80:** mnemônicos → opcodes (incluindo prefixos `CB`, `ED`, `DD`/`FD` para `IX`/`IY`) — trabalho mecânico mas grande, geralmente ~700-900 entradas com modos de endereçamento
- **Formato objeto próprio:** bytes + tabela de relocations (símbolo, offset, tipo: absoluto 16-bit, relativo 8-bit para `JR`/`DJNZ`, etc.)

- **Linker:** resolve símbolos entre units + runtime, aloca endereços finais, resolve relocations, gera o `.COM`

Esse é provavelmente o maior bloco de trabalho “burro” do projeto — vale eu já separar isso como um sub-projeto independente e testável (você poderia inclusive usá-lo depois para outras coisas, tipo o fMSX port ou o msxbasica).

4. Modelo de memória: 128KB com bank switching

MSX2 com mapper trabalha em **páginas de 16KB** (4 slots visíveis em `0000h-3FFFh`, `4000h-7FFFh`, `8000h-BFFFh`, `C000h-FFFFh`), com bancos de RAM trocáveis via portas `A8h-ABh` (ou via API do mapper na BIOS/MSX-DOS 2, mais portátil entre implementações de mapper).

Implicações pro compilador/linker:

- **Área comum fixa** (geralmente `4000h-7FFFh` ou similar) contendo runtime + código “sempre presente” — units grandes ou menos usadas ficam em bancos pagináveis
- **Chamadas cross-bank precisam de trampolim:** não dá pra fazer `CALL` direto pra endereço em outro banco, porque ele pode não estar mapeado. Padrão usado (LROM/MegaROM games fazem isso): uma rotina fixa na área comum que troca o banco, salva o banco anterior, faz o `CALL`, restaura o banco ao retornar
- O linker precisa saber, pra cada símbolo, **em qual banco ele mora**, e decidir automaticamente se emite `CALL` direto (mesmo banco) ou `CALL` via trampolim (banco diferente)

Isso é a parte conceitualmente mais delicada do projeto inteiro — é onde “melhor que o TP4 original” realmente se justifica, porque o TP4 nunca precisou lidar com bank switching.

5. Runtime library (RTL)

Equivalente à `System` unit do TP: I/O de arquivo via BDOS do MSX-DOS (compatível CP/M), `WriteLn/ReadLn` de console, gerenciamento de heap (`New/Dispose`) dentro do banco de dados. Depois, uma unit tipo `CRT/Graph` usando BIOS/VRAM do MSX2 pra tela — aí sim ganhando bastante sobre o TP3/4 originais em termos de recursos gráficos.

Próximo passo sugerido

Dado o tamanho, eu recomendaria atacar nessa ordem: **(1) assembler Z80 em Go** (testável isoladamente, gera `.COM` a partir de assembly puro) → **(2) linker com suporte a bank switching** → só depois **(3) o compilador Pascal em si**. Assim você valida o esqueleto Z80/mapper antes de investir na frente de linguagem.

Quer que eu comece desenhando o formato do objeto próprio e a tabela de encoding Z80, ou prefere primeiro fechar o design do trampolim de bank switching?

Turno 3

| Wilson

Alem do Pascal, que outra linguagem voce acha que seria legal criar um compilador para MSX 2 com MSX-DOS e 128Kb de RAM mapeada? Pascal...legal, mas daria para fazer o MSX-BASIC compilado? um C ? so um assembler? Fortran? FORTH ? LISP? um dBASE novo? o que traria resultados mais rapidos usando um PC para gerar o .COM final para o MSX? sugere outra linguagem?

| Claude

Ótima pergunta — dá pra pensar nisso em termos de “trabalho de frontend/semântica” vs “trabalho de runtime”, porque isso é o que realmente determina velocidade de resultado, não a “fama” da linguagem. Minha opinião, ranqueada por velocidade real de entrega de algo funcional:

1. Assembler puro (Z80) — o vencedor óbvio, e você já vai precisar dele

Nem é bem uma “linguagem alternativa”, é o degrau zero do seu próprio plano do Pascal. Sem frontend de linguagem, sem semântica de tipos — só lexer de mnemônicos + encoder + seu linker com bank switching. Dá resultado utilizável em dias, e depois **vira a base de tudo o mais** (Forth, Pascal, o que for). Eu literalmente publicaria isso sozinho como ferramenta antes de qualquer linguagem de alto nível.

2. FORTH — minha recomendação de “linguagem de verdade” mais rápida

Isso não é força de hábito nostálgico, é arquitetural: Forth historicamente é **a** linguagem mais rápida de bootstrapar em micros 8-bit porque o compilador é trivial — um kernel pequeníssimo de primitivas em Z80 (threaded code, DTC ou ITC) e **o resto do sistema é escrito no próprio Forth**, incluindo o compilador incremental (: ;). Não existe fase de “parsing de gramática complexa com AST e sema” — é dicionário de words + interpretação de tokens.

Por que combina bem com o que você já decidiu:

- **Bank switching:** o modelo de dicionário do Forth (blocos de código encadeados por endereço) mapeia naturalmente pra segmentação em bancos — sistemas Forth de época já lidavam com isso em cartuchos bancados
- **Interativo:** você ganha um REPL rodando direto no MSX via serial ou tela, ótimo pra demos e pra comunidade retro
- Existe precedente histórico real de Forth no MSX, então há público interessado

Contras honestos: sintaxe postfix assusta muita gente, e “vender” um Forth pro público geral do MSX é mais nichado que Pascal ou BASIC.

3. LISP (interpretador, não compilador) — segunda opção rápida

Um interpretador Lisp simples (sem compilar pra Z80 nativo, só um avaliador de S-expressions rodando dentro do .COM) também é rápido de montar — menos que Forth, mas ainda muito menos trabalho que um compilador com checagem de tipos estática. O calo aqui é GC e desempenho: em 128KB com Z80 a ~3.5MHz, um Lisp ingênuo fica perceptivelmente lento pra uso sério, ao contrário do Forth que roda quase na velocidade do assembly.

4. BASIC compilado — tentador pelo seu histórico, mas enganosamente grande

Você já tem o `msxbasic`, então parece o caminho natural. Mas “compilar” MSX-BASIC de verdade (não só tokenizar) exige replicar fidelidade de runtime gigante: strings dinâmicas, `PLAY / SOUND`, comandos gráficos de tela, `GOSUB / GOTO` livre, tipagem fraca com coerção implícita. A linguagem em si é simples, mas a **superfície de compatibilidade** é enorme se você quer que programas MSX-BASIC reais rodem sem reescrever. Resultado rápido só se você aceitar um subset moderno (tipo um “BASIC estruturado”, sem replicar 100% do dialeto original).

5. C — eu pularia

Você já usa SDCC pro projeto MSXgl. Escrever seu próprio compilador C seria redundante com uma ferramenta madura que você já domina, e a complexidade (ponteiros, structs, preprocessor) é comparável ao Pascal — sem ganho de velocidade de entrega.

6. FORTRAN — baixo interesse

Tecnicamente não é mais rápido que um subset de Pascal (semântica parecida, sem ponteiros/units), mas o público retro-MSX pra Fortran é praticamente zero. Eu não priorizaria.

7. dBASE novo — o mais lento de todos, apesar da sintaxe simples

A linguagem em si (comandos tipo `USE`, `LOCATE`, `REPORT FORM`) é fácil de parsear, mas o trabalho pesado está no **runtime**: formato de arquivo `.DBF` próprio, indexação (B-tree/ISAM), motor de tela orientado a formulário. É essencialmente “escrever um banco de dados” antes de “escrever uma linguagem”. Deixaria pro final, se algum dia.

Sugestão fora da sua lista: COMAL

Vale conhecer se você não conhece: COMAL (Common Algorithmic Language) foi um dialeto **estruturado** de BASIC (procedures, `IF-THEN-ELSE-ENDIF`, variáveis locais, sem números de linha obrigatórios) muito popular no ensino escandinavo nos anos 80 — e **teve porte oficial pro MSX** (COMAL 80). É basicamente o meio-termo entre a acessibilidade do BASIC e a disciplina do Pascal, com precedente histórico real na plataforma. Compilar (não interpretar) um COMAL seria um projeto de porte parecido com o Pascal-subset mínimo, mas com apelo mais amplo que Forth/Lisp.

Minha recomendação prática de ordem: assembler (já necessário) → **Forth** (resultado mais rápido e mais “wow” pra comunidade retro, além de servir de laboratório pro bank switching antes do Pascal) → aí sim o Pascal/TP4 que vocês já desenharam.

Turno 4

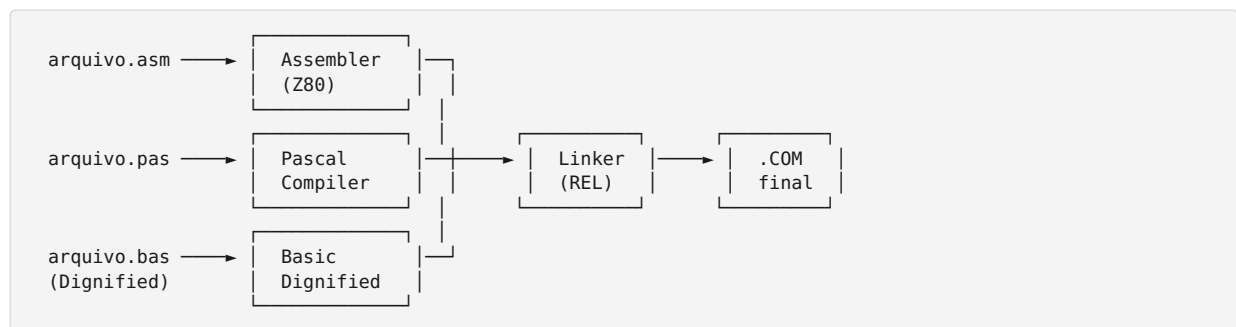
Wilson

voce consegue me ajudar a fazer um assembler que gere codigo relocavel, juntamente com o Pascal gerando relocaves tambem (arquivos .OBJ ou .REL) e que depois consiga linkar em um COM so? daria para por o Basic Dignified no mesmo pacote? assim eu poderia escrever um programa em Assembly, MSX-BASIC Dignified Suite e Pascal, e linkar tudo em um unico .COM ?

Claude

Isso é totalmente possível, e o mais interessante: você já tem uma peça enorme desse quebra-cabeça pronta. O `msxbasica` tem um **assembler/linker Z80 compatível com o formato REL do M80/L80 da Microsoft**, validado byte a byte contra as ferramentas reais MSX (N80/LK80/LB80). Isso é exatamente o tipo de “formato objeto comum” que você precisa pra linkar múltiplas linguagens num `.COM` só — é literalmente pra isso que o formato REL/M80 foi desenhado historicamente (Fortran-80, Cobol-80, Macro-80 todos linkavam junto usando esse mesmo formato nos anos 70/80 da CP/M).

Arquitetura proposta



Cada frontend gera um `.REL` (ou `.OBJ`, mesmo conceito) com símbolos `PUBLIC/EXTRN`, e um único linker resolve tudo — igual o Macro-80/Fortran-80/Cobol-80 faziam na época.

O que precisa ficar bem definido: a ABI comum

Isso é o ponto crítico que separa “dá pra linkar” de “dá pra chamar função de uma linguagem de dentro da outra de forma sã”:

- **Convenção de chamada:** como parâmetros são passados (pilha? registradores HL/DE/BC? um misto, como o Turbo Pascal original fazia com alguns parâmetros em registrador)
- **Convenção de retorno:** onde fica o valor de retorno (A pra byte, HL pra word/ponteiro, etc.)
- **Quem limpa a pilha:** caller ou callee (isso muda o código gerado em toda chamada)
- **Representação de strings:** Pascal usa string com tamanho no byte 0 (`length-prefixed`), MSX-BASIC também usa algo parecido internamente, mas os *runtimes* são diferentes — então strings cruzando a fronteira de linguagem precisam de uma representação comum ou de rotinas de conversão explícitas
- **Segmentos:** REL do M80 já tem o conceito de `CSEG/DSEG/ASEG` — pra bank switching, isso provavelmente vira segmentos nomeados por banco (algo como `BANK0_CSEG`, `BANK1_CSEG`), com o linker decidindo alocação e inserindo trampolins nas chamadas cross-bank

Duas decisões que eu preciso da sua opinião antes de detalhar o formato REL e a ABI

[Pergunta de esclarecimento 1] Formato objeto/linker: reaproveitar o assembler/linker REL do `msxbasica` (PureBasic, já validado) como base, ou reimplementar do zero em Go seguindo a mesma spec REL/M80? Opções: Reusar o do `msxbasica` (PureBasic) como está / Reimplementar em Go seguindo a mesma spec REL / Formato totalmente novo, não REL

[Pergunta de esclarecimento 2] Convenção de chamada entre linguagens: seguir o padrão do Turbo Pascal original (alguns parâmetros em registrador), ou definir uma ABI própria mais simples e uniforme

(tudo via pilha, por exemplo)? Opções: Estilo Turbo Pascal (registradores) / ABI própria, tudo via pilha /
Ainda não sei, quero discutir

Turno 5

Wilson

P: Formato objeto/linker: reaproveitar o assembler/linker REL do msxbasica (PureBasic, já validado) como base, ou reimplementar do zero em Go seguindo a mesma spec REL/M80? R: Formato totalmente novo, não REL

P: Convenção de chamada entre linguagens: seguir o padrão do Turbo Pascal original (alguns parâmetros em registrador), ou definir uma ABI própria mais simples e uniforme (tudo via pilha, por exemplo)? R: ABI própria, tudo via pilha

Claude

Perfeito, isso simplifica bastante e me dá liberdade pra desenhar o formato já pensando em bank switching desde o dia zero (o REL do M80 nunca precisou lidar com isso). Vou propor um design concreto.

1. Formato de objeto (vou chamar de `.MOB` — MSX Object)

Header:

Offset	Tamanho	Campo
0	4	Magic "MOB1"
4	1	Versão do formato
5	2	Nº de segmentos
7	2	Nº de símbolos
9	2	Nº de relocations
11	2	Offset da tabela de strings (nomes de símbolos)

Tabela de segmentos (um por trecho de código/dado):

Tipo: CODE | DATA | BSS
Banco: 0 = área comum (fixa), 1..N = banco paginável
Tamanho
Bytes brutos (CODE/DATA) – BSS só reserva espaço

Tabela de símbolos:

Nome (índice na tabela de strings)
Classe: PUBLIC (exportado) | EXTERN (importado)
Kind: PROC | DATA ← importante pro linker decidir se precisa trampolim
Segmento + offset (se PUBLIC)

Tabela de relocations, cada entrada:

Segmento + offset onde aplicar
Símbolo alvo
Tipo: ABS16 | REL8 | BANKNUM (byte com o nº do banco do símbolo, usado em código que faz paginação manual)

REL8 (usado por JR/DJNZ) só é permitido dentro do mesmo segmento/banco — cross-bank sempre vira CALL/JP absoluto via trampolim, nunca salto relativo.

2. Modelo de banco (proposta de mapa de memória)

0000h-3FFFh	Página 0	Fixa – reservada p/ MSX-DOS/BIOS
4000h-7FFFh	Página 1	Fixa – runtime comum + código "sempre presente"
8000h-BFFFh	Página 2	JANELA COMUTÁVEL – onde entram os bancos pagináveis
C000h-FFFFh	Página 3	Fixa – pilha, heap, buffers

Todo código de unit/banco grande é linkado pra rodar dentro da janela de 16KB da página 2; o linker decide qual banco cada segmento ocupa e gera os trampolins automaticamente.

Trampolim gerado pelo linker (fica na área comum, página 1) pra cada símbolo PUBLIC que mora em

banco diferente do chamador:

```
CALL_simbolo:
    push af
    in  a,(MAPPER_PAGE2) ; salva banco atual da página 2
    push af
    ld  a, N              ; banco onde o símbolo mora
    out (MAPPER_PAGE2), a
    call real_simbolo     ; endereço já dentro de 8000h-BFFFh
    pop af
    out (MAPPER_PAGE2), a ; restaura banco anterior
    pop af
    ret
```

Se chamador e chamado estão no mesmo banco, o linker emite `CALL` direto sem trampolim — otimização automática.

3. ABI própria (tudo via pilha)

- **Parâmetros:** empilhados esquerda→direita (primeiro parâmetro fica mais fundo na pilha)
- **Frame pointer:** `IX`, prólogo padrão:

```
push ix
ld  ix, 0
add ix, sp ; IX agora aponta pro topo do frame
```

- **Quem limpa a pilha:** o **chamador**, logo após o `CALL` (Z80 não tem `RET n` como x86, então limpar no callee exigiria instruções extras toda vez que a função retorna — mais barato deixar o chamador ajustar `SP` uma vez por chamada):

```
ld  hl, N ; N = bytes dos parâmetros empilhados
add hl, sp
ld  sp, hl
```

- **Retorno:** A pra byte/boolean, `HL` pra word/ponteiro. Tipos maiores (records, strings longas) retornam via **parâmetro oculto**: o chamador aloca o espaço e passa o endereço como argumento extra — mesmo truque que compiladores C/Pascal usam até hoje
- **Strings entre linguagens:** formato neutro comum — 1 byte de tamanho + até 255 bytes de dados (Pascal short string tradicional). Assembly, Pascal e Basic Dignified todos leem/escrevem esse mesmo layout, evitando conversão nas fronteiras

Próximo passo

Com isso desenhado, dá pra começar a codificar de forma incremental e testável. Minha sugestão de ordem de implementação em Go:

1. **Structs do formato** `.MOB` + serialização/deserialização (leitura e escrita) — base de tudo
2. **Assembler Z80 mínimo** que já emite `.MOB` (começando só com um subconjunto de mnemônicos pra validar o pipeline ponta a ponta)
3. **Linker** que resolve um único banco (sem trampolim ainda) → gera `.COM` simples
4. Só depois, adicionar suporte a múltiplos bancos + trampolins automáticos

Quer que eu já comece criando os arquivos Go com as structs do formato `.MOB` e o esqueleto do assembler, ou prefere primeiro fechar a tabela completa de opcodes Z80 que o assembler vai precisar cobrir?

Turno 6

Wilson

antes, so de brincadeira, crie um programa hipotetico, a cola seria um programa em Pascal, ele iniciaria, chame rotina para ajustar a tela para SCREEN 2, color 15,1,1, key off, width 32 e key off feita em assembly, faca uma entrada de dados qualquer, em seguida trace um grafico simples feito em MSX-BASIC, retorne ao pascal e aguarde o usuario finalizar. Nao precisa fazer o programa completo, quero so um croqui, uma ideia de como ficaria depois de tudo implementado, imagine que o Pascal vai usar uma fonte propria de 2Kb e vai ter uma tela de abertura em 12Kb, e que a rotina em Assembly vai ter outras coisas e vai ocupar 12Kb, assim como o programa em basic vai ocupar mais uns 5Kb, o proprio programa em pascal ocuparia uns 12Kb. Gere o fonte hipotetico de tudo, imagine que criamos um utilitario tipo MAKE ou JUST e crie este utilitario que vai ser a receita de criacao do .COM final, use nomes interessantes para os compiladores de assembly, Pascal e Basic, alem de nomes interessantes para o linker e para um possivel utilitario de biblioteca, mostre apenas superficialmente como isso funcionaria, so uma prova de conceito, so para eu ter a visao de como ficaria apos tudo implementado (nao precisa compilar nada, so montar o Makefile mesmo) assim temos um exemplo pratico a seguir

Claude

Adorei a proposta — vai ficar ótimo pra visualizar o pipeline completo. Vou usar nomes com o mesmo espírito “1980s Japonês” do banner que você já tinha criado pro IDE.

A “caixa de ferramentas”:

- **KAJI80** — assembler Z80 (*kaji* = ferreiro/forjador, forja o código de máquina)
- **WIRTH80** — compilador Pascal (homenagem a Niklaus Wirth)
- **DIGNAC** — compilador do BASIC Dignified
- **MUSUBI** — linker (*musubi* = “atar/amarrar”, liga os módulos)
- **HA KO** — bibliotecário/empacotador de .mob (*hako* = “caixa”)
- **OBI** — orquestrador de build (*obi* = “faixa/cinto”, amarra a receita inteira) — arquivo de receita: Obifile

main.pas — o cola (WIRTH80, banco 0, ~12Kb)

```
program Demo;

{$USES ScreenFX in 'screen.asm'} { KAJI80, banco 1 }
{$USES Chart   in 'chart.bas'}   { DIGNAC,  banco 2 }
{$RESOURCE Font in 'font.fnt'}   { 2Kb, fonte própria, banco 2 }

var
  valor: Integer;

begin
  ScreenFX.Setup;                { SCREEN 2, COLOR 15,1,1, WIDTH 32, KEY OFF }
  Write('Informe um valor: ');
  ReadLn(valor);
  Chart.Desenhar(valor);         { salta pro módulo BASIC Dignified }
  WriteLn('Pressione uma tecla para sair...');
  ReadKey;
end.
```

screen.asm — ajuste de tela + outras rotinas (KAJI80, banco 1, ~12Kb)

```

MODULE SCREENFX
BANK 1

PUBLIC Setup
EXTERN BIOS_CHGMOD, BIOS_CHGCLR, BIOS_WIDTH, BIOS_KEYOFF

Setup:
    ld    a, 2
    call  BIOS_CHGMOD      ; SCREEN 2
    ld    a, 15
    ld    b, 1
    ld    c, 1
    call  BIOS_CHGCLR      ; COLOR 15,1,1
    ld    a, 32
    call  BIOS_WIDTH       ; WIDTH 32
    call  BIOS_KEYOFF      ; KEY OFF
    ret

; ... demais rotinas do módulo (efeitos, sprites, etc – resto dos ~12Kb)

ENDMOD

```

chart.bas — gráfico simples (DIGNAC, banco 2, ~5Kb)

```

' chart.bas – modulo BASIC Dignified
MODULE Chart
    PUBLIC Desenhlar

    PROCEDURE Desenhlar(valor%)
        LOCAL x%, y%
        LINE (0,0)-(255,191), 1, BF
        FOR x% = 0 TO 255
            y% = 191 - (x% MOD (valor% + 1)) * 2
            PSET (x%, y%), 15
        NEXT x%
    END PROCEDURE
END MODULE

```

obifile — a receita (lida pelo OBI)

```

target: demo.com
entry: Main.Demo

resources:
- file: opening.scr
  bank: 3
  size: 12K
  desc: "Tela de abertura SC2"

- file: font.fnt
  bank: 2
  size: 2K
  desc: "Fonte propria do Pascal"

modules:
- name: Main
  source: main.pas
  compiler: wirth80
  bank: 0      # area comum, sempre mapeada
  size: 12K

- name: ScreenFX
  source: screen.asm
  compiler: kaji80
  bank: 1
  size: 12K

- name: Chart
  source: chart.bas
  compiler: dignac
  bank: 2
  size: 5K

link:
tool: musubi
map: demo.map      # relatorio de enderecos + trampolins gerados
trampoline: auto   # MUSUBI decide sozinho onde precisa

library:
tool: hako
archive: runtime.hlib # runtime comum (I/O, strings, etc) empacotado

```

O comando final

```

$ obi build
KAJI80    screen.asm    -> screen.mob    (banco 1, 12K)
DIGNAC    chart.bas     -> chart.mob     (banco 2, 5K)
WIRTH80   main.pas      -> main.mob      (banco 0, 12K)
HAKO      runtime.hlib  [ja empacotada]
MUSUBI    linkando 3 modulos + runtime.hlib...
           gerando trapolim: Main -> Chart (banco 0 -> banco 2)
           gerando trapolim: Main -> ScreenFX (banco 0 -> banco 1)
           mapa de memoria: demo.map
-> demo.com (43K distribuidos em 4 bancos)

```

Dá pra ver no `Obifile` exatamente a ideia final: cada linguagem no seu banco, `MUSUBI` decidindo os trampolins automaticamente pela diferença de banco entre chamador e chamado, e o `Obi` orquestrando tudo com uma sintaxe declarativa (bem no espírito de um `Makefile/justfile` moderno, mas pensado pra bancos de memória em vez de dependências de arquivo).

Quando você quiser sair do croqui e começar a implementar de verdade, seguimos pela ordem que combinamos: structs do `.MOB` → `KAJI80` mínimo → `MUSUBI` sem trapolim → trapolim multi-banco.

Turno 7

| Wilson

pode criar um documento estilo MD com tudo o que discutimos aqui? quero colocar isso no github como proposta do projeto total, e se possível crie um diretório demo, e nele crie os programas em asm, pas, bas além do makefile só para termos uma ideia do como vai ficar. crie como SPEC.md, depois crie um README.md com uma ideia geral resumida do projeto, crie um changelog, um manual e um release notes, vamos manter esta versão como 0.0.0 do projeto, crie um pacote com todos documentos e diretórios e demo, e crie um pacote ZIP com tudo fácil para baixar. E sugira um nome legal para o projeto, na linha dos módulos que você fez que eu gostei muito, e o pacote fica -v.0.0.0.zip, já crie o repositório zip, e já de um commit com uma frase bacana de efeito, e já gere um nome bem legal para este “release” mesmo sendo só uma ideia, um conceito

| Claude

O projeto se chama **KIZUNA** (絆, “laço/vínculo”) — em sintonia com o MUSUBI, o linker do próprio toolchain. A release ficou registrada como **v0.0.0 “Hatsumusubi”** (初結び, “o primeiro nó amarrado”).

O que foi feito:

- `git init` local, todos os arquivos commitados com a mensagem de efeito, e tag `v0.0.0` criada
- SPEC.md — especificação técnica completa de tudo que discutimos (formato .MOB, modelo de bancos, trampolim, ABI)
- README.md — visão geral resumida
- MANUAL.md — rascunho de uso da toolchain e do Obifile
- CHANGELOG.md e RELEASE_NOTES.md — histórico e notas desta primeira release conceitual
- demo/ COM `main.pas`, `screen.asm`, `chart.bas` e um `Obifile` de exemplo, exatamente o croqui que montamos antes

O ZIP acima já está pronto pra você extrair e dar `git remote add origin ...` direto num repositório novo no GitHub (o histórico do git ficou só local por enquanto, o ZIP tem os arquivos limpos, sem `.git`, pra facilitar o download).

(Arquivo entregue: `kizuna-v0.0.0.zip`)

Turno 8

Wilson

voce pode gerar um PDF com toda esta conversa? desde o inicio ate o final,na integra, sem reumos, sem cortes?

Claude

(Esta resposta — a geração deste próprio PDF.)